09A - 2-Layer Experiment: Jester Pretrain -> IPN Fine-tune

This notebook mirrors the transfer-learning workflow from notebook 08, but targets IPN instead of LD-CONG:

1. Phase 1: Pretrain (or reuse checkpoint) on Jester
2. Phase 2: Fine-tune on IPN
3. Evaluate final IPN test accuracy and macro-F1
4. Rank the resulting TwoStreamLSTM accuracy against published IPN baselines

In [3]:
```
import copy
import json
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import torch

REPO_ROOT = Path.cwd().resolve().parent if Path.cwd().name == "notebooks" el
sys.path.insert(0, str(REPO_ROOT / "src"))

plt.rcParams.update(
    {
        "figure.dpi": 110,
        "figure.facecolor": "white",
        "axes.spines.top": False,
        "axes.spines.right": False,
    }
)
sns.set_palette("tab10")

if torch.cuda.is_available():
    DEVICE = torch.device("cuda")
elif torch.backends.mps.is_available():
    DEVICE = torch.device("mps")
else:
    DEVICE = torch.device("cpu")

print(f"Repository root : {REPO_ROOT}")
print(f"PyTorch version : {torch.__version__}")
print(f"CUDA available  : {torch.cuda.is_available()}")
print(f"Device          : {DEVICE}")
```

```
Repository root : /home/divclab/.Neil/Maestro
PyTorch version : 2.6.0+cu124
CUDA available  : True
Device          : cuda
```

1. Configuration

In [4]:

```python
# Config and dataset paths
JESTER_CONFIG_PATH = REPO_ROOT / "configs/model/two_stream_attn_jester_2laye
JESTER_NPZ = REPO_ROOT / "data/processed/jester_pretrain_dataset.npz"
IPN_NPZ = REPO_ROOT / "data/processed/ipn_dataset_v6_w32.npz"

CHECKPOINT_DIR = REPO_ROOT / "models/checkpoints"
RUN_ROOT_DIR = REPO_ROOT / "experiments/runs"

# Reuse latest Jester pretrain checkpoint when available; otherwise Phase 1
_JESTER_CKPT_CANDIDATES = sorted(
    [
        p
        for p in CHECKPOINT_DIR.glob("two_stream_attn_v1_2layer_pretrain_*.p
        if "_finetune_" not in p.name
    ],
    key=lambda p: p.stat().st_mtime if p.exists() else 0,
    reverse=True,
)
PRETRAINED_CHECKPOINT = _JESTER_CKPT_CANDIDATES[0] if _JESTER_CKPT_CANDIDATE

# Stage schedule
FREEZE_EPOCHS = 8
STAGE_B_WARMUP_EPOCHS = 5
BACKBONE_LR_MULTIPLIER = 0.1

# Continual-learning retention terms are disabled for this direct IPN compar
FORGETTING_PENALTY_WEIGHT = 0.0
DISTILLATION_WEIGHT = 0.0
REPLAY_CE_WEIGHT = 0.0
REPLAY_MAX_SAMPLES_PER_CLASS = 500
GPM_COMPONENTS = 0

# Optional override: uncomment to force a specific pretrain checkpoint.
# PRETRAINED_CHECKPOINT = REPO_ROOT / "models/checkpoints/two_stream_attn_v1

required = [JESTER_CONFIG_PATH, JESTER_NPZ, IPN_NPZ]
missing = [p for p in required if not p.exists()]
if missing:
    raise FileNotFoundError(
        "Missing required files:\n" + "\n".join(f"  - {p}" for p in missing)
    )

print(f"Jester config        : {JESTER_CONFIG_PATH.name}")
print(f"Jester dataset       : {JESTER_NPZ.name}")
print(f"IPN dataset          : {IPN_NPZ.name}")
print(f"Checkpoint directory : {CHECKPOINT_DIR}")
```

```
print(f"Run root directory   : {RUN_ROOT_DIR}")
print(f"Reuse pretrain ckpt  : {PRETRAINED_CHECKPOINT}")
```

```
Jester config        : two_stream_attn_jester_2layer_experiment.yaml
Jester dataset       : jester_pretrain_dataset.npz
IPN dataset          : ipn_dataset_v6_w32.npz
Checkpoint directory : /home/divclab/.Neil/Maestro/models/checkpoints
Run root directory   : /home/divclab/.Neil/Maestro/experiments/runs
Reuse pretrain ckpt  : None
```

2. Load Configs and Build IPN Fine-tuning Config

In [9]:

```
from maestro.training.train_lstm import TrainingPipelineConfig, load_trainir

jester_config: TrainingPipelineConfig = load_training_config(JESTER_CONFIG_F

with np.load(JESTER_NPZ, allow_pickle=True) as raw_jester:
    jester_labels = raw_jester["labels"].astype(str)
    jester_metadata = json.loads(str(raw_jester["metadata_json"].item())) if
    jester_feature_dim = int(jester_metadata.get("feature_dim", raw_jester["
    jester_window = int(jester_metadata.get("window_length", raw_jester["sec

jester_vocab = list(jester_metadata.get("vocabulary", sorted(set(jester_labe
jester_num_classes = len(jester_vocab)

# Runtime NPZ metadata is the source of truth for this experiment.
jester_config.model.input_size = jester_feature_dim
jester_config.model.num_classes = jester_num_classes

with np.load(IPN_NPZ, allow_pickle=True) as raw_ipn:
    ipn_labels_all = raw_ipn["labels"].astype(str)
    ipn_rec_ids_all = raw_ipn.get("recording_ids", np.array(["?"] * len(ipn_
    ipn_metadata = json.loads(str(raw_ipn["metadata_json"].item())) if "meta
    ipn_feature_dim = int(ipn_metadata.get("feature_dim", raw_ipn["sequences
    ipn_window = int(ipn_metadata.get("window_length", raw_ipn["sequences"].

# For fair comparison against published IPN baselines, exclude unknown backg
excluded_ipn_classes = {"unknown"}
excluded_lc = {c.lower() for c in excluded_ipn_classes}
keep_mask = np.array([label.lower() not in excluded_lc for label in ipn_labe

ipn_labels = ipn_labels_all[keep_mask]
ipn_rec_ids = ipn_rec_ids_all[keep_mask]

if ipn_labels.size == 0:
    raise ValueError("All IPN samples were filtered out after excluding clas

ipn_vocab_raw = list(ipn_metadata.get("vocabulary", sorted(set(ipn_labels_al
ipn_vocab = [c for c in ipn_vocab_raw if c.lower() not in excluded_lc]
if not ipn_vocab:
    ipn_vocab = sorted(set(ipn_labels.tolist()))

# Rebuild contiguous label ids after class filtering.
ipn_label_to_idx = {label: idx for idx, label in enumerate(ipn_vocab)}
```

```
unknown_labels_in_samples = sorted(set(ipn_labels.tolist()) - set(ipn_label_
if unknown_labels_in_samples:
    raise ValueError(
        "Filtered IPN labels contain classes not present in vocabulary after
        f"{unknown_labels_in_samples}"
    )
ipn_label_ids = np.array([ipn_label_to_idx[label] for label in ipn_labels],

ipn_num_classes = len(ipn_vocab)

ipn_config: TrainingPipelineConfig = copy.deepcopy(jester_config)
ipn_config.model.model_name = "two_stream_attn_v1_2layer_ipn"
ipn_config.model.input_size = ipn_feature_dim
ipn_config.model.num_classes = ipn_num_classes

# Keep datamodule loading in finetune_lstm consistent with this notebook fil
ipn_config.training.exclude_classes = sorted(excluded_ipn_classes)

# For direct comparison to published baselines, keep weighting disabled.
ipn_config.training.class_weighting = False
ipn_config.training.max_samples_per_class = None
ipn_config.training.epochs = max(20, ipn_config.training.epochs)

print("Jester pretrain config (runtime NPZ-derived):")
print(f"  model_name   : {jester_config.model.model_name}")
print(f"  num_classes  : {jester_config.model.num_classes}")
print(f"  input_size   : {jester_config.model.input_size}")
print(f"  window_length: {jester_window}")
print()
print("IPN fine-tune config (unknown excluded):")
print(f"  model_name   : {ipn_config.model.model_name}")
print(f"  num_classes  : {ipn_config.model.num_classes}")
print(f"  input_size   : {ipn_config.model.input_size}")
print(f"  epochs       : {ipn_config.training.epochs}")
print(f"  class_weight : {ipn_config.training.class_weighting}")
print(f"  exclude_cls  : {ipn_config.training.exclude_classes}")
print()
print("IPN dataset filtering:")
print(f"  excluded classes  : {sorted(excluded_ipn_classes)}")
print(f"  samples kept      : {int(keep_mask.sum()):,} / {len(keep_mask):,}"
print(f"  removed samples   : {int((~keep_mask).sum()):,}")
print()
print("IPN dataset metadata:")
print(f"  window_length: {ipn_window}")
print(f"  feature_dim  : {ipn_feature_dim}")
print(f"  classes      : {ipn_num_classes}")
```

```
Jester pretrain config (runtime NPZ-derived):
  model_name   : two_stream_attn_v1_2layer_pretrain
  num_classes  : 20
  input_size   : 147
  window_length: 32

IPN fine-tune config (unknown excluded):
  model_name   : two_stream_attn_v1_2layer_ipn
  num_classes  : 13
  input_size   : 147
  epochs       : 80
  class_weight : False
  exclude_cls  : ['unknown']

IPN dataset filtering:
  excluded classes  : ['unknown']
  samples kept      : 383,811 / 394,824
  removed samples   : 11,013

IPN dataset metadata:
  window_length: 32
  feature_dim  : 147
  classes      : 13
```

3. Quick IPN Split and Class Distribution Check

In [6]:

```python
from collections import Counter

from maestro.training.datamodule import GestureDataModule

dm_ipn = GestureDataModule(
    dataset_path=IPN_NPZ,
    batch_size=ipn_config.training.batch_size,
    split=ipn_config.split,
    seed=ipn_config.training.seed,
    num_workers=0,
    pin_memory=False,
    max_samples_per_class=ipn_config.training.max_samples_per_class,
    exclude_classes=ipn_config.training.exclude_classes,
)
dm_ipn.setup_indices_only(label_ids=ipn_label_ids, recording_ids=ipn_rec_ids

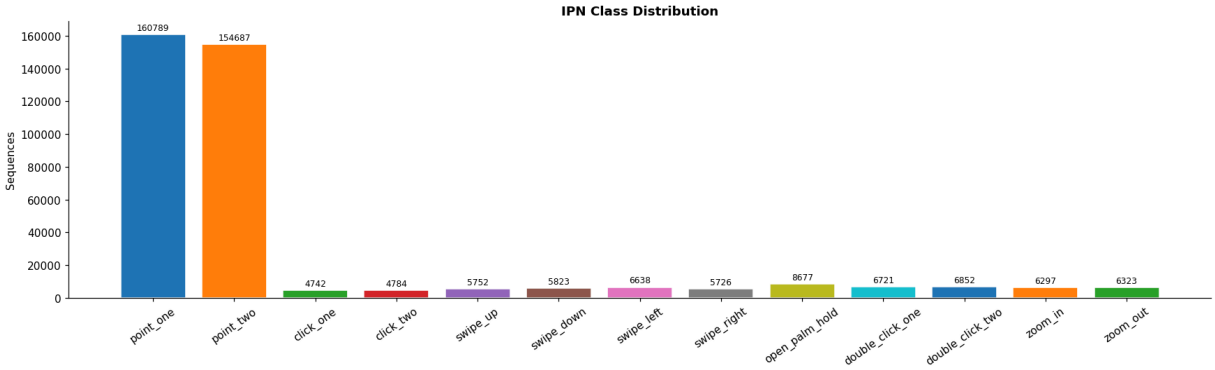
print("Split sizes:")
print(f"  train={len(dm_ipn.train_indices):,}  val={len(dm_ipn.val_indices):
print(f"  num_classes={dm_ipn.num_classes}")

dist_all = Counter(ipn_labels.tolist())
class_labels = dm_ipn.class_labels
counts_all = [dist_all.get(c, 0) for c in class_labels]

fig, ax = plt.subplots(figsize=(16, 5))
bars = ax.bar(class_labels, counts_all, color=sns.color_palette("tab10", ler
ax.bar_label(bars, fmt="%d", fontsize=8, padding=2)
ax.set_title("IPN Class Distribution", fontweight="bold")
ax.set_ylabel("Sequences")
```

```
ax.tick_params(axis="x", rotation=35)
plt.tight_layout()
plt.show()
```

```
Split sizes:
  train=323,082  val=36,058  test=24,671
  num_classes=13
```



4. Phase 1 - Jester Pretraining (or checkpoint reuse)

In [7]:
```
from maestro.training.train_lstm import TrainingResult, train_lstm

if PRETRAINED_CHECKPOINT is not None and PRETRAINED_CHECKPOINT.exists():
    print(f"Reusing existing Jester checkpoint: {PRETRAINED_CHECKPOINT.name}
    pretrain_result = None
else:
    print("Running Phase 1 Jester pretraining from scratch...")
    pretrain_result: TrainingResult = train_lstm(
        dataset_path=JESTER_NPZ,
        config=jester_config,
        output_dir=CHECKPOINT_DIR,
        run_root_dir=RUN_ROOT_DIR,
        device=DEVICE,
    )
    PRETRAINED_CHECKPOINT = pretrain_result.checkpoint_path

print()
print(f"Pretrained checkpoint to use for IPN fine-tuning: {PRETRAINED_CHECKP
```

```
Running Phase 1 Jester pretraining from scratch...

Training  two_stream_attn_v1_2layer_pretrain_20260517T025154Z
  Model     : EnhancedTwoStreamLSTM  (1,210,484 params)
  Dataset   : train=41,071  val=4,835
  Optimiser : AdamW  lr=1.00e-04  wd=5.00e-04
  Schedule  : ReduceLROnPlateau  factor=0.5  patience=8
  Stopping  : patience=15  min_delta=1e-04  metric=val_macro_f1
--------------------------------------------------------------------------------
---------
 Epoch  TrainLoss  TrainAcc   ValLoss   ValAcc   ValF1        LR    Time
ES  Note
--------------------------------------------------------------------------------
---------
     1/80       2.8506     20.3%    1.9283    42.4%   55.1%  1.00e-04    4.8
s  * best
     2/80       1.8803     54.5%    1.6407    67.0%   73.7%  1.00e-04    4.6
s  * best
     3/80       1.7057     63.6%    1.5497    71.6%   77.0%  1.00e-04    4.6
s  * best
     4/80       1.6165     68.0%    1.4897    76.1%   80.7%  1.00e-04    4.6
s  * best
     5/80       1.5551     71.3%    1.4599    80.0%   82.8%  1.00e-04    4.5
s  * best
     6/80       1.5155     74.4%    1.4297    82.3%   84.4%  1.00e-04    4.6
s  * best
     7/80       1.4799     76.8%    1.4195    81.0%   83.9%  1.00e-04    4.6
s    ES 1/15
     8/80       1.4537     79.1%    1.3949    83.8%   85.4%  1.00e-04    4.5
s  * best
     9/80       1.4339     80.6%    1.3912    84.7%   85.8%  1.00e-04    4.7
s  * best
    10/80       1.4145     82.3%    1.3855    84.7%   86.0%  1.00e-04    4.6
s  * best
    11/80       1.3998     83.6%    1.3698    86.1%   86.2%  1.00e-04    4.7
s  * best
    12/80       1.3892     84.4%    1.3687    88.6%   87.8%  1.00e-04    4.6
s  * best
    13/80       1.3799     85.4%    1.3609    87.1%   86.9%  1.00e-04    4.7
s    ES 1/15
    14/80       1.3710     86.3%    1.3551    87.8%   87.5%  1.00e-04    4.5
s    ES 2/15
    15/80       1.3633     86.9%    1.3503    87.8%   87.5%  1.00e-04    4.5
s    ES 3/15
    16/80       1.3503     87.8%    1.3507    88.6%   88.0%  1.00e-04    4.5
s  * best
    17/80       1.3447     88.2%    1.3463    89.2%   87.8%  1.00e-04    4.6
s    ES 1/15
    18/80       1.3431     88.8%    1.3364    88.8%   88.0%  1.00e-04    4.5
s  * best
    19/80       1.3347     89.1%    1.3471    89.8%   88.5%  1.00e-04    4.6
s  * best
    20/80       1.3327     89.8%    1.3343    90.2%   88.5%  1.00e-04    4.6
s  * best
    21/80       1.3242     90.1%    1.3395    89.5%   88.5%  1.00e-04    4.5
s    ES 1/15
```

```
    22/80       1.3216     90.4%    1.3303    90.4%   88.7%  1.00e-04    4.7
s  * best
    23/80       1.3170     90.7%    1.3387    90.4%   88.7%  1.00e-04    4.6
s  * best
    24/80       1.3120     91.1%    1.3378    90.9%   89.1%  1.00e-04    4.5
s  * best
    25/80       1.3078     91.4%    1.3352    90.7%   88.8%  1.00e-04    4.4
s    ES 1/15
    26/80       1.3054     91.5%    1.3306    91.2%   89.4%  1.00e-04    4.5
s  * best
    27/80       1.3028     91.9%    1.3314    91.1%   89.3%  1.00e-04    4.6
s    ES 1/15
    28/80       1.2989     92.2%    1.3332    91.0%   89.1%  1.00e-04    4.4
s    ES 2/15
    29/80       1.2960     92.3%    1.3214    91.5%   89.3%  1.00e-04    4.4
s    ES 3/15
    30/80       1.2933     92.6%    1.3217    90.8%   89.2%  1.00e-04    4.5
s    ES 4/15
    31/80       1.2877     92.8%    1.3332    91.6%   89.3%  1.00e-04    4.5
s    ES 5/15
    32/80       1.2840     93.0%    1.3277    91.5%   89.5%  1.00e-04    4.5
s  * best
    33/80       1.2847     93.3%    1.3305    90.9%   89.1%  1.00e-04    4.7
s    ES 1/15
    34/80       1.2811     93.3%    1.3370    91.6%   89.6%  1.00e-04    4.6
s  * best
    35/80       1.2756     93.5%    1.3264    91.3%   89.4%  1.00e-04    4.6
s    ES 1/15
    36/80       1.2783     93.6%    1.3223    91.5%   89.6%  1.00e-04    4.5
s  * best
    37/80       1.2737     93.7%    1.3294    91.7%   89.7%  1.00e-04    4.6
s  * best
    38/80       1.2690     94.0%    1.3375    91.9%   89.6%  1.00e-04    4.6
s    ES 1/15
    39/80       1.2672     94.1%    1.3282    91.7%   89.5%  1.00e-04    4.5
s    ES 2/15
    40/80       1.2662     94.2%    1.3404    92.2%   89.7%  1.00e-04    4.5
s  * best
    41/80       1.2637     94.4%    1.3319    92.0%   89.8%  1.00e-04    4.6
s  * best
    42/80       1.2587     94.7%    1.3287    91.5%   89.4%  1.00e-04    4.7
s    ES 1/15
    43/80       1.2605     94.6%    1.3444    91.5%   89.5%  1.00e-04    4.6
s    ES 2/15
    44/80       1.2561     94.8%    1.3366    92.0%   89.8%  1.00e-04    4.6
s  * best
    45/80       1.2550     94.9%    1.3464    92.2%   89.8%  1.00e-04    4.7
s  * best
    46/80       1.2548     94.9%    1.3410    91.7%   89.6%  1.00e-04    4.6
s    ES 1/15
    47/80       1.2531     95.0%    1.3552    91.7%   89.4%  1.00e-04    4.5
s    ES 2/15
    48/80       1.2498     95.2%    1.3500    92.0%   89.8%  1.00e-04    4.5
s  * best
    49/80       1.2459     95.3%    1.3406    91.8%   89.6%  1.00e-04    4.5
s    ES 1/15
```

```
    50/80       1.2441     95.4%    1.3496    91.4%   89.4%  1.00e-04    4.6
s    ES 2/15
    51/80       1.2453     95.5%    1.3566    92.0%   89.7%  1.00e-04    4.5
s    ES 3/15
    52/80       1.2428     95.6%    1.3569    92.2%   89.7%  1.00e-04    4.5
s    ES 4/15
    53/80       1.2429     95.7%    1.3405    91.5%   89.6%  1.00e-04    4.5
s    ES 5/15
    54/80       1.2392     95.8%    1.3668    90.9%   89.0%  1.00e-04    4.5
s    ES 6/15
    55/80       1.2422     95.8%    1.3640    91.6%   89.4%  1.00e-04    4.5
s    ES 7/15
    56/80       1.2367     95.9%    1.3679    91.6%   89.4%  1.00e-04    4.5
s    ES 8/15
    57/80       1.2349     96.0%    1.3631    91.9%   89.5%  5.00e-05    4.5
s    ES 9/15
    58/80       1.2302     96.4%    1.3597    92.0%   89.7%  5.00e-05    4.6
s    ES 10/15
    59/80       1.2278     96.5%    1.3692    91.6%   89.4%  5.00e-05    4.5
s    ES 11/15
    60/80       1.2288     96.5%    1.3654    91.5%   89.5%  5.00e-05    4.5
s    ES 12/15
    61/80       1.2252     96.5%    1.3643    91.3%   89.4%  5.00e-05    4.5
s    ES 13/15
    62/80       1.2243     96.7%    1.3686    91.7%   89.5%  5.00e-05    4.5
s    ES 14/15
    63/80       1.2249     96.6%    1.3720    91.9%   89.6%  5.00e-05    4.5
s    ES 15/15

Early stopping at epoch 63  (no improvement in val_macro_f1 for 15 epochs)

Best epoch  : 48  val_macro_f1=0.8983  val_loss=1.3500

Pretrained checkpoint to use for IPN fine-tuning: /home/divclab/.Neil/Maestr
o/models/checkpoints/two_stream_attn_v1_2layer_pretrain_20260517T025154Z.pt
```

5. Phase 2 - IPN Fine-tuning (Stage A + Stage B)

In [10]:

```python
from maestro.training.train_lstm import finetune_lstm

if PRETRAINED_CHECKPOINT is None:
    raise RuntimeError("No pretrained checkpoint available. Run Phase 1 firs

finetune_result: TrainingResult = finetune_lstm(
    dataset_path=IPN_NPZ,
    config=ipn_config,
    pretrained_checkpoint_path=PRETRAINED_CHECKPOINT,
    output_dir=CHECKPOINT_DIR,
    run_root_dir=RUN_ROOT_DIR,
    device=DEVICE,
    freeze_epochs=FREEZE_EPOCHS,
    stage_b_warmup_epochs=STAGE_B_WARMUP_EPOCHS,
    replay_dataset_path=None,
```

```
    replay_max_samples_per_class=REPLAY_MAX_SAMPLES_PER_CLASS,
    distillation_weight=DISTILLATION_WEIGHT,
    replay_ce_weight=REPLAY_CE_WEIGHT,
    backbone_lr_multiplier=BACKBONE_LR_MULTIPLIER,
    forgetting_penalty_weight=FORGETTING_PENALTY_WEIGHT,
    gpm_components=GPM_COMPONENTS,
)

print()
print(f"Fine-tuned checkpoint: {finetune_result.checkpoint_path}")
print(f"Run dir            : {finetune_result.run_dir}")
```

```
Fine-tuning  two_stream_attn_v1_2layer_ipn_finetune_20260517T030213Z
  Pretrained : two_stream_attn_v1_2layer_pretrain_20260517T025154Z.pt
  Model      : EnhancedTwoStreamLSTM  (1,209,133 params)
  Dataset    : train=323,082  val=36,058
  Classes    : 13  ['point_one', 'point_two', 'click_one', 'click_two', 'swi
pe_up', 'swipe_down', 'swipe_left', 'swipe_right', 'open_palm_hold', 'double
_click_one', 'double_click_two', 'zoom_in', 'zoom_out']

──────────────────────────────────────────────────────────────────────────────
────
Stage A — backbone frozen, head-only  (8 epochs, lr=1e-3)
------------------------------------------------------------------------------
----
 Epoch  TrainLoss  TrainAcc   ValLoss   ValAcc   ValF1
------------------------------------------------------------------------------
----
     1/8        0.6089     89.6%    0.5361    91.9%   69.0%
     2/8        0.5279     92.4%    0.5268    92.3%   70.4%
     3/8        0.5051     93.3%    0.5194    92.7%   70.9%
     4/8        0.4916     93.9%    0.5231    92.4%   70.9%
     5/8        0.4817     94.2%    0.5204    92.8%   71.6%
     6/8        0.4739     94.5%    0.5240    92.7%   70.2%
     7/8        0.4680     94.7%    0.5217    92.6%   71.5%
     8/8        0.4629     94.9%    0.5242    92.8%   70.9%

Stage A complete. Val macro-F1 = 0.7092

──────────────────────────────────────────────────────────────────────────────
──────────────────────────────────
Stage B — full fine-tuning  (up to 80 epochs, head_lr=1.00e-04, backbone_lr=
1.00e-05  [warmup 5 epochs: 0→target])
------------------------------------------------------------------------------
----------------------------------
 Epoch  TrainLoss  TrainAcc   ValLoss   ValAcc   ValF1   JstrF1        LR
Distill  ReplayCE  Note
------------------------------------------------------------------------------
----------------------------------
     1/80       0.4463     95.6%    0.5137    92.9%   71.8%      nan  1.00e-
04       nan       nan  * best
     2/80       0.4402     95.8%    0.5100    93.1%   72.5%      nan  1.00e-
04       nan       nan  * best
     3/80       0.4333     96.1%    0.5071    93.2%   72.8%      nan  1.00e-
04       nan       nan  * best
     4/80       0.4274     96.3%    0.5052    93.3%   73.5%      nan  1.00e-
04       nan       nan  * best
     5/80       0.4219     96.5%    0.4997    93.5%   73.9%      nan  1.00e-
04       nan       nan  * best
     6/80       0.4163     96.8%    0.5025    93.4%   74.0%      nan  1.00e-
04       nan       nan  * best
     7/80       0.4115     96.9%    0.4977    93.7%   73.9%      nan  1.00e-
04       nan       nan    ES 1/15
     8/80       0.4078     97.1%    0.5001    93.7%   74.3%      nan  1.00e-
04       nan       nan  * best
     9/80       0.4035     97.2%    0.4996    93.7%   74.8%      nan  1.00e-
04       nan       nan  * best
    10/80       0.3997     97.4%    0.4978    93.8%   74.9%      nan  1.00e-
```

```
04       nan       nan  * best
    11/80       0.3968     97.5%    0.4972    93.9%   75.2%      nan  1.00e-
04       nan       nan  * best
    12/80       0.3936     97.6%    0.4952    93.9%   74.5%      nan  1.00e-
04       nan       nan    ES 1/15
    13/80       0.3900     97.8%    0.4973    94.0%   75.0%      nan  1.00e-
04       nan       nan    ES 2/15
    14/80       0.3879     97.9%    0.4943    94.1%   75.6%      nan  1.00e-
04       nan       nan  * best
    15/80       0.3853     97.9%    0.4974    94.0%   75.4%      nan  1.00e-
04       nan       nan    ES 1/15
    16/80       0.3828     98.0%    0.4950    94.1%   75.5%      nan  1.00e-
04       nan       nan    ES 2/15
    17/80       0.3807     98.1%    0.4941    94.2%   76.3%      nan  1.00e-
04       nan       nan  * best
    18/80       0.3777     98.2%    0.4947    94.1%   75.8%      nan  1.00e-
04       nan       nan    ES 1/15
    19/80       0.3756     98.3%    0.4948    94.2%   76.2%      nan  1.00e-
04       nan       nan    ES 2/15
    20/80       0.3735     98.4%    0.4945    94.3%   76.9%      nan  1.00e-
04       nan       nan  * best
    21/80       0.3719     98.4%    0.4909    94.3%   77.0%      nan  1.00e-
04       nan       nan  * best
    22/80       0.3699     98.5%    0.4942    94.3%   77.1%      nan  1.00e-
04       nan       nan  * best
    23/80       0.3681     98.6%    0.4933    94.4%   77.5%      nan  1.00e-
04       nan       nan  * best
    24/80       0.3666     98.6%    0.4902    94.4%   77.3%      nan  1.00e-
04       nan       nan    ES 1/15
    25/80       0.3645     98.7%    0.4921    94.4%   77.6%      nan  1.00e-
04       nan       nan  * best
    26/80       0.3629     98.8%    0.4948    94.3%   77.0%      nan  1.00e-
04       nan       nan    ES 1/15
    27/80       0.3617     98.8%    0.4936    94.4%   77.1%      nan  1.00e-
04       nan       nan    ES 2/15
    28/80       0.3603     98.8%    0.4911    94.4%   77.5%      nan  1.00e-
04       nan       nan    ES 3/15
    29/80       0.3590     98.9%    0.4897    94.4%   77.3%      nan  1.00e-
04       nan       nan    ES 4/15
    30/80       0.3571     98.9%    0.4910    94.6%   78.6%      nan  1.00e-
04       nan       nan  * best
    31/80       0.3561     99.0%    0.4905    94.5%   77.7%      nan  1.00e-
04       nan       nan    ES 1/15
    32/80       0.3549     99.0%    0.4908    94.5%   77.2%      nan  1.00e-
04       nan       nan    ES 2/15
    33/80       0.3535     99.1%    0.4901    94.5%   77.7%      nan  1.00e-
04       nan       nan    ES 3/15
    34/80       0.3525     99.1%    0.4896    94.6%   77.8%      nan  1.00e-
04       nan       nan    ES 4/15
    35/80       0.3511     99.2%    0.4921    94.4%   77.1%      nan  1.00e-
04       nan       nan    ES 5/15
    36/80       0.3500     99.2%    0.4897    94.6%   78.3%      nan  1.00e-
04       nan       nan    ES 6/15
    37/80       0.3492     99.2%    0.4921    94.6%   78.0%      nan  1.00e-
04       nan       nan    ES 7/15
    38/80       0.3483     99.2%    0.4909    94.5%   78.4%      nan  1.00e-
```

```
04       nan       nan    ES 8/15
    39/80       0.3476     99.2%    0.4893    94.6%   78.2%      nan  5.00e-
05       nan       nan    ES 9/15
    40/80       0.3455     99.3%    0.4877    94.6%   78.4%      nan  5.00e-
05       nan       nan    ES 10/15
    41/80       0.3448     99.3%    0.4885    94.6%   78.1%      nan  5.00e-
05       nan       nan    ES 11/15
    42/80       0.3446     99.3%    0.4888    94.7%   78.8%      nan  5.00e-
05       nan       nan  * best
    43/80       0.3442     99.4%    0.4886    94.6%   78.6%      nan  5.00e-
05       nan       nan    ES 1/15
    44/80       0.3437     99.4%    0.4897    94.6%   78.0%      nan  5.00e-
05       nan       nan    ES 2/15
    45/80       0.3432     99.4%    0.4895    94.6%   78.5%      nan  5.00e-
05       nan       nan    ES 3/15
    46/80       0.3433     99.4%    0.4888    94.6%   78.2%      nan  5.00e-
05       nan       nan    ES 4/15
    47/80       0.3425     99.4%    0.4888    94.6%   78.3%      nan  5.00e-
05       nan       nan    ES 5/15
    48/80       0.3421     99.4%    0.4878    94.5%   78.3%      nan  5.00e-
05       nan       nan    ES 6/15
    49/80       0.3415     99.4%    0.4882    94.7%   78.5%      nan  5.00e-
05       nan       nan    ES 7/15
    50/80       0.3411     99.5%    0.4901    94.6%   78.0%      nan  5.00e-
05       nan       nan    ES 8/15
    51/80       0.3406     99.4%    0.4879    94.7%   78.2%      nan  2.50e-
05       nan       nan    ES 9/15
    52/80       0.3400     99.5%    0.4885    94.7%   78.7%      nan  2.50e-
05       nan       nan    ES 10/15
    53/80       0.3399     99.5%    0.4866    94.7%   78.1%      nan  2.50e-
05       nan       nan    ES 11/15
    54/80       0.3396     99.5%    0.4876    94.6%   78.1%      nan  2.50e-
05       nan       nan    ES 12/15
    55/80       0.3392     99.5%    0.4872    94.6%   78.2%      nan  2.50e-
05       nan       nan    ES 13/15
    56/80       0.3388     99.5%    0.4859    94.6%   78.2%      nan  2.50e-
05       nan       nan    ES 14/15
    57/80       0.3387     99.5%    0.4870    94.6%   78.4%      nan  2.50e-
05       nan       nan    ES 15/15

Early stopping at epoch 57  (no improvement for 15 epochs)

Best epoch : 42  val_macro_f1=0.7879  composite=0.7879  val_loss=0.4888

Fine-tuned checkpoint: /home/divclab/.Neil/Maestro/models/checkpoints/two_st
ream_attn_v1_2layer_ipn_finetune_20260517T030213Z.pt
Run dir            : /home/divclab/.Neil/Maestro/experiments/runs/two_stream
_attn_v1_2layer_ipn_finetune_20260517T030213Z
```

6. Learning Curves

In [11]:

```python
history = finetune_result.train_history
epochs = [h["epoch"] for h in history]
stage_a_count = len([h for h in history if h.get("stage", 1) == 0])
boundary_epoch = epochs[stage_a_count - 1] + 0.5 if stage_a_count > 0 else N
```

```
fig, axes = plt.subplots(1, 3, figsize=(18, 4.8))

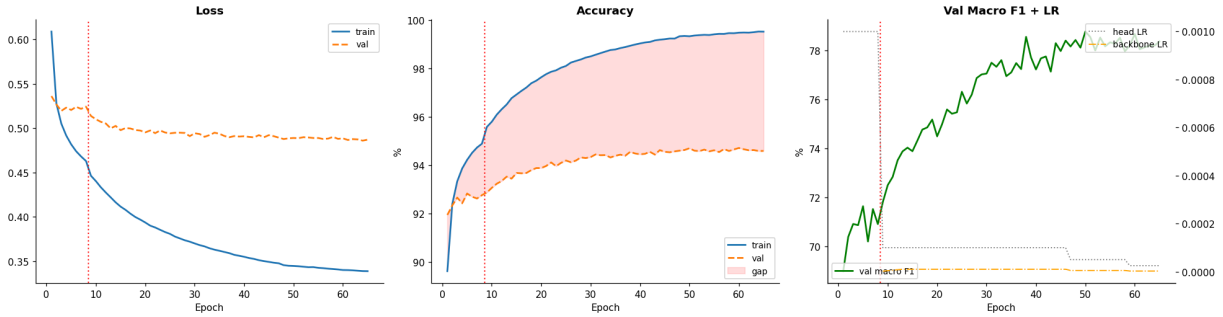
ax = axes[0]
ax.plot(epochs, [h["train_loss"] for h in history], label="train", linewidth
ax.plot(epochs, [h["val_loss"] for h in history], label="val", linewidth=1.8
if boundary_epoch:
    ax.axvline(boundary_epoch, color="red", linestyle=":", linewidth=1.5, al
ax.set_title("Loss", fontweight="bold")
ax.set_xlabel("Epoch")
ax.legend(fontsize=9)

ax = axes[1]
train_acc = [h["train_accuracy"] * 100 for h in history]
val_acc = [h["val_accuracy"] * 100 for h in history]
ax.plot(epochs, train_acc, label="train", linewidth=1.8)
ax.plot(epochs, val_acc, label="val", linewidth=1.8, linestyle="--")
ax.fill_between(epochs, val_acc, train_acc, alpha=0.12, color="red", label="
if boundary_epoch:
    ax.axvline(boundary_epoch, color="red", linestyle=":", linewidth=1.5, al
ax.set_title("Accuracy", fontweight="bold")
ax.set_xlabel("Epoch")
ax.set_ylabel("%")
ax.legend(fontsize=9)

ax = axes[2]
ax.plot(
    epochs,
    [h.get("val_macro_f1", float("nan")) * 100 for h in history],
    label="val macro F1",
    linewidth=1.8,
    color="green",
)
if boundary_epoch:
    ax.axvline(boundary_epoch, color="red", linestyle=":", linewidth=1.5, al
ax2 = ax.twinx()
ax2.plot(
    epochs,
    [h.get("learning_rate", float("nan")) for h in history],
    label="head LR",
    linewidth=1.2,
    color="gray",
    linestyle=":",
)
ax2.plot(
    [h["epoch"] for h in history if h.get("stage", 1) == 1],
    [h.get("backbone_lr", float("nan")) for h in history if h.get("stage", 1
    label="backbone LR",
    linewidth=1.2,
    color="orange",
    linestyle="-.",
)
ax.set_title("Val Macro F1 + LR", fontweight="bold")
ax.set_xlabel("Epoch")
ax.set_ylabel("%")
ax.legend(fontsize=9, loc="lower left")
```

```
ax2.legend(fontsize=9, loc="upper right")

plt.tight_layout()
plt.show()
```



7. IPN Test Metrics

In [12]:
```
ipn_test = finetune_result.test_report

print("=" * 68)
print("PHASE 2 TEST RESULTS (IPN)")
print("=" * 68)
print(f"Accuracy   : {ipn_test.accuracy:.1%}")
print(f"Macro F1   : {ipn_test.macro_f1:.1%}")
print(f"Loss       : {ipn_test.loss:.4f}")
print(f"ECE        : {ipn_test.calibration_ece:.4f}")
```

```
====================================================================
PHASE 2 TEST RESULTS (IPN)
====================================================================
Accuracy   : 97.2%
Macro F1   : 83.0%
Loss       : 0.3807
ECE        : 0.0421
```

8. Baseline Ranking by Accuracy

This section compares your trained TwoStreamLSTM result against the IPN paper baseline rows you listed.

In [13]:
```
BASELINES = [
    ("ResNeXt-101", "RGB-Flow", "86.32%"),
    ("ResNeXt-101", "RGB-Seg", "84.77%"),
    ("ResNeXt-101", "RGB", "83.59%"),
    ("C3D", "RGB", "77.75%"),
    ("ResNet-50", "RGB-Seg", "75.11%"),
    ("ResNet-50", "RGB-Flow", "74.65%"),
    ("ResNet-50", "RGB", "73.10%"),
]

rows = [
    {
        "Model": model,
```

```
        "Modality": modality,
        "Accuracy": acc,
        "Accuracy_num": float(acc.strip("%")),
    }
    for model, modality, acc in BASELINES
]

rows.append(
    {
        "Model": "TwoStreamLSTM (this run)",
        "Modality": "Skeleton",
        "Accuracy": f"{ipn_test.accuracy * 100:.2f}%",
        "Accuracy_num": ipn_test.accuracy * 100.0,
    }
)

df_rank = pd.DataFrame(rows).sort_values("Accuracy_num", ascending=False).re
df_rank.insert(0, "Rank", np.arange(1, len(df_rank) + 1))

display(df_rank[["Rank", "Model", "Modality", "Accuracy"]])

our_rank = int(df_rank[df_rank["Model"] == "TwoStreamLSTM (this run)"].index
best_acc = float(df_rank.iloc[0]["Accuracy_num"])
gap_to_top = best_acc - ipn_test.accuracy * 100.0

print()
print(f"TwoStreamLSTM rank: {our_rank}/{len(df_rank)}")
if gap_to_top <= 0:
    print("TwoStreamLSTM is currently top-ranked by accuracy in this table."
else:
    print(f"Gap to top baseline: {gap_to_top:.2f} percentage points")
```

	Rank	Model	Modality	Accuracy
0	1	TwoStreamLSTM (this run)	Skeleton	97.24%
1	2	ResNeXt-101	RGB-Flow	86.32%
2	3	ResNeXt-101	RGB-Seg	84.77%
3	4	ResNeXt-101	RGB	83.59%
4	5	C3D	RGB	77.75%
5	6	ResNet-50	RGB-Seg	75.11%
6	7	ResNet-50	RGB-Flow	74.65%
7	8	ResNet-50	RGB	73.10%

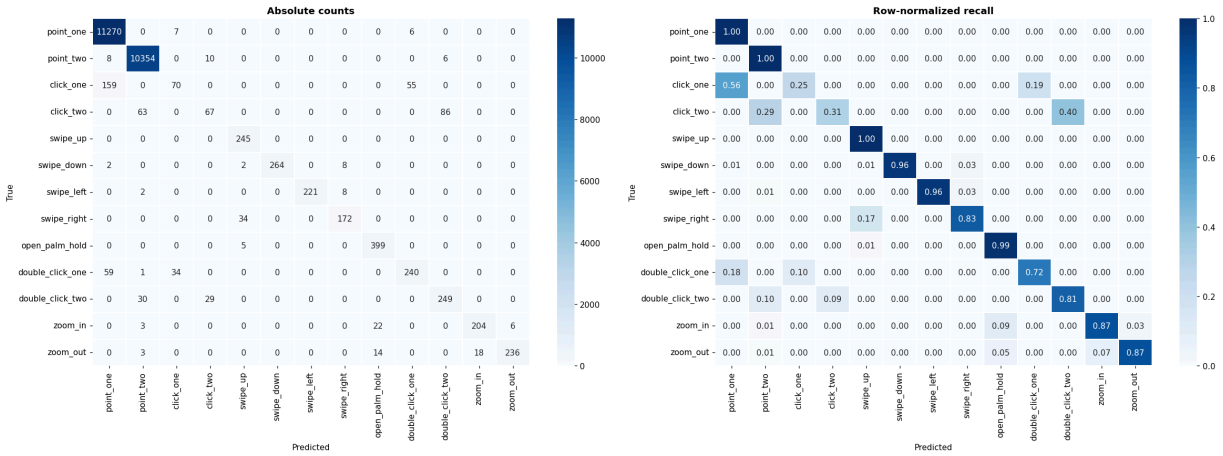
```
TwoStreamLSTM rank: 1/8
TwoStreamLSTM is currently top-ranked by accuracy in this table.
```

9. Confusion Matrix (IPN Test)

In [14]:
```
cm = np.array(ipn_test.confusion_matrix, dtype=np.float32)
cm_norm = cm / (cm.sum(axis=1, keepdims=True) + 1e-9)
```

```
fig, axes = plt.subplots(1, 2, figsize=(22, 8))
for ax, matrix, title, fmt in [
    (axes[0], cm, "Absolute counts", ".0f"),
    (axes[1], cm_norm, "Row-normalized recall", ".2f"),
]:
    sns.heatmap(
        matrix,
        annot=True,
        fmt=fmt,
        cmap="Blues",
        xticklabels=finetune_result.class_labels,
        yticklabels=finetune_result.class_labels,
        linewidths=0.3,
        ax=ax,
        cbar=True,
    )
    ax.set_title(title, fontweight="bold")
    ax.set_xlabel("Predicted")
    ax.set_ylabel("True")
    ax.tick_params(axis="x", rotation=90)
    ax.tick_params(axis="y", rotation=0)

plt.tight_layout()
plt.show()
```



10. Final Outputs

In [15]:
```
print("=" * 68)
print("FINAL OUTPUTS")
print("=" * 68)
print(f"Pretrained checkpoint : {PRETRAINED_CHECKPOINT}")
print(f"Fine-tuned checkpoint: {finetune_result.checkpoint_path}")
print(f"Run directory         : {finetune_result.run_dir}")
print(f"Model version         : {finetune_result.model_version}")
print()
print(f"IPN test accuracy     : {ipn_test.accuracy:.1%}")
print(f"IPN test macro F1     : {ipn_test.macro_f1:.1%}")
print(f"IPN test ECE          : {ipn_test.calibration_ece:.4f}")
```

```
======================================================================
FINAL OUTPUTS
======================================================================
Pretrained checkpoint : /home/divclab/.Neil/Maestro/models/checkpoints/two_s
tream_attn_v1_2layer_pretrain_20260517T025154Z.pt
Fine-tuned checkpoint: /home/divclab/.Neil/Maestro/models/checkpoints/two_st
ream_attn_v1_2layer_ipn_finetune_20260517T030213Z.pt
Run directory        : /home/divclab/.Neil/Maestro/experiments/runs/two_stre
am_attn_v1_2layer_ipn_finetune_20260517T030213Z
Model version        : two_stream_attn_v1_2layer_ipn_finetune_20260517T03021
3Z

IPN test accuracy    : 97.2%
IPN test macro F1    : 83.0%
IPN test ECE         : 0.0421
```